```
import pandas as pd
import numpy as np

# Raw retail customer export
raw_data = {
    'cust_id': [101, 102, 102, 103, 104],
    'name': [' Alice ', 'Bob', 'Bob', 'charlie', 'David '],
    'age': [25, np.nan, np.nan, 30, 22]
}
df = pd.DataFrame(raw_data)

# Clean: Drop duplicates, strip whitespace, capitalize, impute missing median age
df = df.drop_duplicates(subset=['cust_id']).copy()
df['name'] = df['name'].str.strip().str.capitalize()
df['age'] = df['age'].fillna(df['age'].median())

print(df)
```

Output:

cust_id	name	age
0	101	Alice	25.0
1	102	Bob	25.0
3	103	Charlie	30.0
4	104	David	22.0

```
import pandas as pd
import re

raw_txns = pd.DataFrame({
    'txn_date': ['2026-01-15', '16/01/2026', '2026.01.17'],
    'raw_amount': ['$1,250.50', '€840.00', '$99.99'],
    'phone': ['+1 (555) 019-2834', '555.019.2835', 'Call: 5550192836']
})

# Standardize dates to ISO format
raw_txns['clean_date'] = pd.to_datetime(raw_txns['txn_date'],
format='mixed').dt.strftime('%Y-%m-%d')

# Extract numeric amount (standardizing currency to float)
raw_txns['clean_amount'] = raw_txns['raw_amount'].str.replace(r'[^\d.]', '',
regex=True).astype(float)

# Standardize phone numbers to pure 10-digit format
```

```
raw_txns['clean_phone'] = raw_txns['phone'].apply(lambda x: re.sub(r'\D', '', x)[-10:])

print(raw_txns[['clean_date', 'clean_amount', 'clean_phone']])
```

```
clean_date  clean_amount clean_phone
0  2026-01-15       1250.50  5550192834
1  2026-01-16        840.00  5550192835
2  2026-01-17         99.99  5550192836
```

```
import pandas as pd

orders = pd.DataFrame({
    'order_id': [5001, 5002, 5003],
    'cust_id': [1, 2, 4],
    'item': ['Monitor', 'Keyboard', 'Mouse']
})

customers = pd.DataFrame({
    'cust_id': [1, 2, 3],
    'city': ['Bangalore', 'Mumbai', 'Delhi']
})

# Merge inner to find valid ship orders; identify missing foreign keys via outer
merged = pd.merge(orders, customers, on='cust_id', how='left')
print(merged)
```

```
order_id  cust_id      item       city
0      5001        1   Monitor  Bangalore
1      5002        2  Keyboard     Mumbai
2      5003        4     Mouse        NaN
```

```
import pandas as pd

incoming_batch = pd.DataFrame({
    'sensor_id': ['S1', 'S2', 'S3', 'S4'],
    'reading_celsius': [24.5, -999.0, 31.2, 85.0],  # -999 is sensor error
    'pressure_psi': [14.7, 15.1, 14.9, -2.0]        # -2.0 violates physical bounds
})

def validate_telemetry(df: pd.DataFrame):
    violations = []
    # Rule 1: Temperature bounded between -40C and 60C
    temp_check = df[~df['reading_celsius'].between(-40, 60)]
```

```
    if not temp_check.empty:
        violations.append(f"Invalid Temp at indices: {temp_check.index.tolist()}")

    # Rule 2: Pressure must be strictly positive
    press_check = df[df['pressure_psi'] <= 0]
    if not press_check.empty:
        violations.append(f"Negative Pressure at indices: {press_check.index.tolist()}")

    return violations

issues = validate_telemetry(incoming_batch)
print("Validation Report:", issues)
```

```
Validation Report: ['Invalid Temp at indices: [1, 3]', 'Negative Pressure at indices: [3]']
```

```
import pandas as pd

crm_records = pd.DataFrame({
    'email': ['a@corp.com', 'b@corp.com', None, 'd@corp.com', 'b@corp.com'],
    'phone': ['111', '222', '333', None, '222']
})

completeness = (crm_records['email'].notnull().sum() / len(crm_records)) * 100
uniqueness = (crm_records['email'].nunique() / crm_records['email'].notnull().sum()) * 100

print(f"Completeness KPI: {completeness:.1f}% (Pass: {completeness >= 90})")
print(f"Uniqueness KPI:   {uniqueness:.1f}% (Pass: {uniqueness == 100})")
```

```
—----------------------------------------------------
Completeness KPI: 80.0% (Pass: False)
Uniqueness KPI:   75.0% (Pass: False)
—----------------------------------------------------
```

```
import pandas as pd

df = pd.DataFrame({
    'account_id': [1001, 1002, 1003, 1004],
    'status': ['ACTIVE', 'INACTIVE', 'PENDING', 'UNKNOWN_STATUS'],
    'balance': [1500.0, 420.50, -50.0, 300.0],
    'email': ['acc1@bank.in', 'acc2@bank.in', None, 'acc4_invalid_email']
})

allowed_statuses = {'ACTIVE', 'INACTIVE', 'PENDING'}
```

```
valid_status_rate = df['status'].isin(allowed_statuses).mean() * 100
valid_balance_rate = (df['balance'] >= 0).mean() * 100
valid_email_rate = df['email'].str.contains(r'^[\w\.-]+@[\w\.-]+\.\w+$', na=False).mean() * 100

dq_scorecard = pd.DataFrame({
    'Dimension': ['Validity (Status)', 'Accuracy (Balance >= 0)', 'Validity (Email Pattern)'],
    'Score (%)': [valid_status_rate, valid_balance_rate, valid_email_rate],
    'SLA Met': [valid_status_rate >= 95, valid_balance_rate >= 99, valid_email_rate >= 90]
})

print(dq_scorecard)
```

```
—----------------------------------------------------------
```

Dimension Score (%) SLA Met
0 Validity (Status) 75.0 False
1 Accuracy (Balance >= 0) 75.0 False
2 Validity (Email Pattern) 50.0 False

```
—----------------------------------------------------------------
import pandas as pd

salaries = pd.Series([35000, 42000, 43000, 45000, 49000, 52000, 150000]) # contains an
outlier

print(f"Mean:   {salaries.mean():.2f}")
print(f"Median: {salaries.median():.2f}")
print(f"StdDev: {salaries.std():.2f}")
print(f"Skew:   {salaries.skew():.2f}") # Positive skew indicates high-end pull
```

```
—---------------------------------------------
```

Mean: 59428.57
Median: 45000.00
StdDev: 40381.11
Skew: 2.36

```
—-------------------------------------------------
import numpy as np
import pandas as pd

np.random.seed(42)
trade_volumes = np.random.normal(loc=1000, scale=150, size=50).tolist() + [3500, 120]
df_trades = pd.DataFrame({'volume': trade_volumes})

Q1 = df_trades['volume'].quantile(0.25)
Q3 = df_trades['volume'].quantile(0.75)
IQR = Q3 - Q1
```

```
lower_fence = Q1 - 1.5 * IQR
upper_fence = Q3 + 1.5 * IQR

anomalies = df_trades[(df_trades['volume'] < lower_fence) | (df_trades['volume'] > upper_fence)]
print(f"IQR: {IQR:.2f} | Threshold: [{lower_fence:.2f}, {upper_fence:.2f}]")
print("Detected Anomalies:\n", anomalies)
```

```
IQR: 184.66 | Threshold: [640.85, 1379.50]
Detected Anomalies:
    volume
50  3500.00
51   120.00
```

```
import pandas as pd

marketing_df = pd.DataFrame({
   'ad_spend': [1000, 2000, 3000, 4000, 5000],
   'clicks': [120, 260, 310, 430, 520],
   'cancellation_rate': [0.12, 0.11, 0.13, 0.10, 0.12]
})

corr_matrix = marketing_df.corr().round(3)
print("Correlation Matrix:\n", corr_matrix)
```

```
Correlation Matrix:
                ad_spend  clicks  cancellation_rate
ad_spend           1.000   0.992             -0.208
clicks             0.992   1.000             -0.347
cancellation_rate -0.208  -0.347              1.000
```

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import numpy as np

features = np.array([
   [10, 200, 0.5, 3],
   [12, 220, 0.6, 4],
   [8, 150, 0.2, 1],
   [45, 800, 0.9, 12],
   [42, 750, 0.85, 11]
])

# 1. Standardize (Mandatory for PCA)
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(features)

# 2. PCA Projection
pca = PCA(n_components=2)
reduced_features = pca.fit_transform(X_scaled)

print("Explained Variance Ratio:", pca.explained_variance_ratio_.round(4))
print("Total Information Retained:", f"{pca.explained_variance_ratio_.sum() * 100:.2f}%")
```

—----------

Explained Variance Ratio: [0.9634 0.0345]
Total Information Retained: 99.79%

—----------

```
import pandas as pd

iteration_log = pd.DataFrame([
   {'iteration': 1, 'changes': 'Raw features + Missing dropped', 'roc_auc': 0.68},
   {'iteration': 2, 'changes': 'Median imputation + Log transform on income', 'roc_auc': 0.74},
   {'iteration': 3, 'changes': 'Iteration 2 + Engineered Debt-to-Income feature', 'roc_auc': 0.81}
])

iteration_log['improvement'] = iteration_log['roc_auc'].diff().fillna(0)
print(iteration_log.to_string(index=False))
```

—----------

```
iteration                                changes  roc_auc  improvement
        1                Raw features + Missing dropped     0.68         0.00
        2     Median imputation + Log transform on income     0.74         0.06
        3  Iteration 2 + Engineered Debt-to-Income feature     0.81         0.07
```

—----------

```
import pandas as pd
from statsmodels.stats.outliers_influence import variance_inflation_factor

X = pd.DataFrame({
   'sqft': [1000, 1500, 2000, 2500, 3000],
   'rooms': [2, 3, 4, 5, 6],
   'heating_area': [990, 1490, 1995, 2480, 2990], # Highly collinear with sqft
   'age': [5, 12, 1, 20, 8]
})

def eliminate_collinear_features(df, thresh=10.0):
   current_df = df.copy()
   while True:
      vif_series = pd.Series(
         [variance_inflation_factor(current_df.values, i) for i in range(current_df.shape[1])],
```

```
            index=current_df.columns
        )
        max_vif = vif_series.max()
        if max_vif > thresh:
            worst_feature = vif_series.idxmax()
            print(f"Pruning '{worst_feature}' (VIF: {max_vif:.2f})")
            current_df = current_df.drop(columns=[worst_feature])
        else:
            break
    return current_df

cleaned_features = eliminate_collinear_features(X)
print("Retained Features:", cleaned_features.columns.tolist())
```

—----------

Pruning 'sqft' (VIF: 35832.89)
Pruning 'heating_area' (VIF: 74.45)
Retained Features: ['rooms', 'age']

—----------

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

X, y = make_classification(n_samples=500, n_features=4, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

clf = LogisticRegression()
clf.fit(X_train, y_train)
preds = clf.predict(X_test)

print("Classification Accuracy:", accuracy_score(y_test, preds))
```

—----------

Pruning 'sqft' (VIF: 35832.89)
Pruning 'heating_area' (VIF: 74.45)
Retained Features: ['rooms', 'age']

—----------

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

X, y = make_classification(n_samples=500, n_features=4, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
clf = LogisticRegression()
clf.fit(X_train, y_train)
preds = clf.predict(X_test)

print("Classification Accuracy:", accuracy_score(y_test, preds))
```
Classification Accuracy: 0.87

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import GridSearchCV
import numpy as np

np.random.seed(42)
X = np.random.rand(100, 5)
y = X[:, 0] * 3 + X[:, 1] * 2 + np.random.randn(100) * 0.1

rfr = RandomForestRegressor(random_state=42)
params = {
    'n_estimators': [10, 50],
    'max_depth': [3, 5]
}

grid = GridSearchCV(rfr, param_grid=params, cv=3, scoring='neg_mean_squared_error')
grid.fit(X, y)

print("Optimal Hyperparameters:", grid.best_params_)
print("Best Cross-Val MSE:", abs(round(grid.best_score_, 4)))
```
Optimal Hyperparameters: {'max_depth': 5, 'n_estimators': 50}
Best Cross-Val MSE: 0.0883

```
import joblib
from sklearn.linear_model import Ridge
import numpy as np

# Train & serialize
model = Ridge().fit(np.array([[1], [2], [3]]), np.array([10, 20, 30]))
joblib.dump(model, 'model.joblib')

# Load back and infer
deployed_model = joblib.load('model.joblib')
pred = deployed_model.predict([[4]])
```

```
print(f"Restored Model Prediction for input [[4]]: {pred[0]:.1f}")
```
Restored Model Prediction for input [[4]]: 40.0

```
import numpy as np
from scipy.stats import ks_2samp

# Historical training distribution
baseline_data = np.random.normal(loc=50.0, scale=5.0, size=1000)

# Live incoming batch with significant distribution shift
live_incoming_data = np.random.normal(loc=54.0, scale=5.2, size=200)

stat, p_value = ks_2samp(baseline_data, live_incoming_data)

print(f"KS Statistic: {stat:.4f} | P-Value: {p_value:.6e}")
if p_value < 0.05:
    print("ALERT: Statistically significant Data Drift detected! Trigger model retraining pipeline.")
else:
    print("Status: Distribution is stable.")
```
KS Statistic: 0.3540 | P-Value: 8.881784e-16
ALERT: Statistically significant Data Drift detected! Trigger model retraining pipeline.

```
import heapq

# Priority Queue (Min-heap: lower number = higher urgency)
task_queue = []
heapq.heappush(task_queue, (3, "Routine daily sync"))
heapq.heappush(task_queue, (1, "CRITICAL: Database failover"))
heapq.heappush(task_queue, (2, "Batch inference request"))

while task_queue:
    priority, job = heapq.heappop(task_queue)
    print(f"Executed: [P{priority}] {job}")
```
Executed: [P1] CRITICAL: Database failover
Executed: [P2] Batch inference request
Executed: [P3] Routine daily sync

```
def find_transaction_cycle(graph):
    visited = set()
    rec_stack = set()

    def dfs(node, path):
        visited.add(node)
        rec_stack.add(node)
        path.append(node)

        for neighbor in graph.get(node, []):
            if neighbor not in visited:
                cycle = dfs(neighbor, path)
                if cycle: return cycle
            elif neighbor in rec_stack:
                cycle_start_idx = path.index(neighbor)
                return path[cycle_start_idx:] + [neighbor]

        rec_stack.remove(node)
        path.pop()
        return None

    for node in graph:
        if node not in visited:
            cycle = dfs(node, [])
            if cycle: return cycle
    return None

# Graph representation: Account A -> Account B
tx_graph = {
    'Acc_A': ['Acc_B'],
    'Acc_B': ['Acc_C'],
    'Acc_C': ['Acc_D'],
    'Acc_D': ['Acc_B']  # Loop back to Acc_B
}

cycle_detected = find_transaction_cycle(tx_graph)
print("Fraud Ring Path Detected:", " -> ".join(cycle_detected))
```

Fraud Ring Path Detected: Acc_B -> Acc_C -> Acc_D -> Acc_B

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# Inputs: [Hours Studied, Attendance % scaled]
X = np.array([0.75, 0.90])
weights = np.array([1.2, 0.8])
bias = -0.5

# Forward Pass
z = np.dot(X, weights) + bias
prediction = sigmoid(z)

print(f"Activation Logit (z): {z:.4f}")
print(f"Pass Probability:    {prediction:.4f}")
```

—----------

Activation Logit (z): 1.1200
Pass Probability: 0.7540

—----------

```
import torch
import torch.nn as nn
import torch.optim as optim

# Sample synthetic inputs (3 features: age, salary, debt) -> 1 binary target
X = torch.tensor([[0.2, 0.8, 0.1], [0.8, 0.1, 0.9], [0.5, 0.5, 0.4]], dtype=torch.float32)
y = torch.tensor([[1.0], [0.0], [1.0]], dtype=torch.float32)

class LoanRiskModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(3, 4)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(4, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        return self.sigmoid(self.fc2(self.relu(self.fc1(x))))

model = LoanRiskModel()
criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters(), lr=0.05)
```

```
# Train loop (3 steps for demonstration)
for epoch in range(3):
    optimizer.zero_grad()
    outputs = model(X)
    loss = criterion(outputs, y)
    loss.backward()
    optimizer.step()
    print(f"Epoch {epoch+1}, Loss: {loss.item():.4f}")
```

Epoch 1, Loss: 0.6974
Epoch 2, Loss: 0.6725
Epoch 3, Loss: 0.6503

```
import numpy as np

class BaseMinMaxScaler:
    def __init__(self):
        self.min_val = None
        self.max_val = None

    def fit(self, x):
        self.min_val = np.min(x)
        self.max_val = np.max(x)
        return self

    def transform(self, x):
        return (x - self.min_val) / (self.max_val - self.min_val)

values = np.array([10.0, 20.0, 30.0, 50.0])
scaler = BaseMinMaxScaler().fit(values)
scaled = scaler.transform(values)

print("Min-Max Transformed Array:", scaled)
```

Min-Max Transformed Array: [0.   0.25 0.5 1.]

```
def optimize_portfolio(budgets, returns, max_capital):
    memo = {}

    def dp(index, current_budget):
        if index == len(budgets) or current_budget <= 0:
            return 0
        state = (index, current_budget)
        if state in memo:
            return memo[state]

        # Choice 1: Skip campaign
        profit_without = dp(index + 1, current_budget)

        # Choice 2: Select campaign if budget allows
        profit_with = 0
        if budgets[index] <= current_budget:
            profit_with = returns[index] + dp(index + 1, current_budget - budgets[index])

        memo[state] = max(profit_without, profit_with)
        return memo[state]

    return dp(0, max_capital)

campaign_costs = [10, 20, 30]
campaign_returns = [60, 100, 120]
budget_limit = 50

max_profit = optimize_portfolio(campaign_costs, campaign_returns, budget_limit)
print(f"Max Portfolio Return achievable under ${budget_limit} limit: ${max_profit}")
```

—----------

Max Portfolio Return achievable under $50 limit: $220

—----------

```
import pandas as pd

audit_log = pd.DataFrame({
    'demographic_group': ['Group_A', 'Group_A', 'Group_B', 'Group_B'],
    'approved': [1, 1, 1, 0]
})

acceptance_rates = audit_log.groupby('demographic_group')['approved'].mean()
disparate_impact = acceptance_rates['Group_B'] / acceptance_rates['Group_A']

print(acceptance_rates)
print(f"\nDisparate Impact Ratio: {disparate_impact:.2f}")
```

```
print("Bias Warning (<0.80):", disparate_impact < 0.80)
```
```
demographic_group
Group_A    1.0
Group_B    0.5
Name: approved, dtype: float64

Disparate Impact Ratio: 0.50
Bias Warning (<0.80): True
```
```
import numpy as np
from sklearn.linear_model import LinearRegression

# Baseline market conditions: X = [interest_rate, debt_ratio] -> Default %
X_normal = np.array([[2.5, 0.20], [3.0, 0.25], [3.5, 0.30], [4.0, 0.35]])
y_normal = np.array([1.2, 1.8, 2.3, 3.1])

stress_model = LinearRegression().fit(X_normal, y_normal)

# Adversarial Scenario: Severe Market Crash (Rates hit 9.5%, debt ratio jumps to 0.85)
extreme_shock = np.array([[9.5, 0.85]])
shock_prediction = stress_model.predict(extreme_shock)[0]

print(f"Baseline Max Expected Default Rate: {y_normal.max()}%")
print(f"Stressed Prediction:              {shock_prediction:.2f}%")
print("Risk Factor Multiplier:           ", f"{shock_prediction / y_normal.max():.2f}x")
```
```
Baseline Max Expected Default Rate: 3.1%
Stressed Prediction:              10.74%
Risk Factor Multiplier:           3.46x
```
```
def map_business_objective_to_metric(business_statement: str):
    mapping = {
        "churn": {"Target": "Recall on positive class", "Threshold": ">= 0.85"},
        "fraud": {"Target": "Precision / Low False Alarms", "Threshold": ">= 0.95"},
        "upsell": {"Target": "Top-decile Lift score", "Threshold": ">= 2.5x"}
    }
    for key, spec in mapping.items():
        if key in business_statement.lower():
            return spec
    return "Custom metric validation needed"

print(map_business_objective_to_metric("Our priority is to decrease customer churn this
quarter"))
```

—----------
{'Target': 'Recall on positive class', 'Threshold': '>= 0.85'}
—----------

```
def generate_stakeholder_report(persona: str, model_metrics: dict):
    if persona == "Executive":
        return {
            "Audience": "C-Suite",
            "Projected Annual Savings": f"${model_metrics['estimated_cost_reduction']:,}",
            "Business Risk Level": "Low" if model_metrics['false_positive_rate'] < 0.05 else
"Moderate"
        }
    elif persona == "Technical_Lead":
        return {
            "Audience": "Data Science / DevOps",
            "ROC_AUC": model_metrics['auc'],
            "P99_Inference_Latency": f"{model_metrics['latency_ms']}ms",
            "F1_Score": model_metrics['f1']
        }

metrics = {"estimated_cost_reduction": 250000, "false_positive_rate": 0.03, "auc": 0.92,
"latency_ms": 14.2, "f1": 0.88}

print("Executive Format:\n", generate_stakeholder_report("Executive", metrics))
print("\nEngineering Format:\n", generate_stakeholder_report("Technical_Lead", metrics))
```

—----------
Executive Format:
{'Audience': 'C-Suite', 'Projected Annual Savings': '$250,000', 'Business Risk Level': 'Low'}

Engineering Format:
{'Audience': 'Data Science / DevOps', 'ROC_AUC': 0.92, 'P99_Inference_Latency': '14.2ms', 'F1_Score': 0.88}

—----------

```
import pandas as pd

sales = pd.DataFrame({
    'region': ['North', 'South', 'North', 'West', 'South', 'North'],
    'revenue': [25000, 15000, 30000, 18000, 22000, 12000]
})

summary = sales.groupby('region')['revenue'].sum().reset_index()
summary['pct_share'] = (summary['revenue'] / summary['revenue'].sum()) * 100
```

```
for _, row in summary.iterrows():
    print(f"Region: {row['region']:5} | Total: ${row['revenue']:,} | Share: {row['pct_share']:.1f}%")
```

```
Region: North | Total: $67,000 | Share: 54.9%
Region: South | Total: $37,000 | Share: 30.3%
Region: West  | Total: $18,000 | Share: 14.8%
```

```
import pandas as pd

churn_data = pd.DataFrame({
    'plan_tier': ['Basic', 'Basic', 'Premium', 'Premium', 'Enterprise', 'Enterprise'],
    'contract_length': ['Monthly', 'Annual', 'Monthly', 'Annual', 'Monthly', 'Annual'],
    'churn_rate': [0.18, 0.04, 0.12, 0.02, 0.08, 0.01]
})

pivot = churn_data.pivot(index='plan_tier', columns='contract_length', values='churn_rate')

def generate_ascii_heatmap(matrix):
    print(f"{'Tier':<12} | {'Annual':<10} | {'Monthly':<10}")
    print("-" * 38)
    for index, row in matrix.iterrows():
        # Highlighting high-risk buckets
        ann_flag = "⚠️ HIGH" if row['Annual'] > 0.10 else "✅ STABLE"
        mon_flag = "⚠️ HIGH" if row['Monthly'] > 0.10 else "✅ STABLE"
        print(f"{index:<12} | {row['Annual']*100:>4.1f}% {ann_flag:<7} | {row['Monthly']*100:>4.1f}%
{mon_flag:<7}")

generate_ascii_heatmap(pivot)
```

```
Tier         | Annual     | Monthly
--------------------------------------
Basic        |  4.0% ✅ STABLE | 18.0% ⚠️ HIGH
Enterprise   |  1.0% ✅ STABLE |  8.0% ✅ STABLE
Premium      |  2.0% ✅ STABLE | 12.0% ⚠️ HIGH
```